Міністерство освіти і науки України

Національний технічний університет України «КПІ імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки

Кафедра інформаційних систем та технологій

ЗВІТ

з лабораторної роботи № 1

з курсу «Технології машинного навчання»

Тема: «Convolutional neural networks»

Перевірив:
викл.
Сарнацький Владислав Віталійович

Виконав:
Студент групи ІА-51мп
Рудяков Ілля

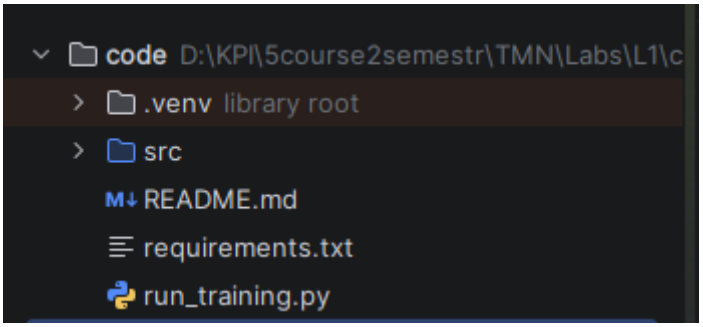
Київ 2026

Лабораторна робота №1

Тема: Convolutional neural networks.

Хід роботи

Для зручності будемо розділяти логіку на файли та директорії



Задамо файл конфігу для всього проекту. Тут основними параметрами є: seed (параметр випадковості), batch_size (розмір одного пакету при обробці пакетами), epochs (кількість епох при навчанні), та train/val_fraction які задають яку частину датасету будемо використовувати (нехай буде 60%).

Створимо окремий файл який міститиме метод set_seed для зміни випадковості в бібліотеках:

```python
import random
import numpy as np
import torch

2 usages
def set_seed(seed: int) -> None:
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)

```

Далі створимо файл для роботи з датасетами. Він міститиме два методи: _make_subset (використання тільки певної частини датасету) та build_loaders (завантаження самого датасету):

```python
2 usages
def _make_subset(ds, fraction: float, seed: int):
    if fraction >= 1.0:
        return ds
    n = len(ds)
    k = max(1, int(n * fraction))
    g = torch.Generator().manual_seed(seed)
    idx = torch.randperm(n, generator=g)[:k].tolist()
    return Subset(ds, idx)
```

```python
2 usages
def build_loaders(cfg: CFG):
    train_tfms = transforms.Compose(
        [
            transforms.RandomCrop( size: 32, padding=4),
            transforms.RandomHorizontalFlip(),
            transforms.ToTensor(),
            transforms.Normalize(MEAN, STD),
        ]
    )
    val_tfms = transforms.Compose(
        [
            transforms.ToTensor(),
            transforms.Normalize(MEAN, STD),
        ]
    )

    full_train = datasets.CIFAR100(
        root=cfg.data_dir, train=True, download=True, transform=train_tfms
    )
    full_val = datasets.CIFAR100(
        root=cfg.data_dir, train=False, download=True, transform=val_tfms
    )

    train_ds = _make_subset(full_train, cfg.train_fraction, cfg.seed)
    val_ds = _make_subset(full_val, cfg.val_fraction, cfg.seed)
```

```python
train_loader = DataLoader(
    train_ds,
    batch_size=cfg.batch_size,
    shuffle=True,
    num_workers=cfg.num_workers,
    pin_memory=torch.cuda.is_available(),
)
val_loader = DataLoader(
    val_ds,
    batch_size=cfg.batch_size,
    shuffle=False,
    num_workers=cfg.num_workers,
    pin_memory=torch.cuda.is_available(),
)

classes: List[str] = full_train.classes
return train_loader, val_loader, classes
```

Тепер перейдемо до самої архітектури моделі. Будемо використовувати архітектуру ResNet. Модель складатиметься з блоків (для цього створимо окремий клас ResidualBlock). Саму модель описуватиме клас CustomResNet

```python
class ResidualBlock(nn.Module):
    def __init__(self, in_ch: int, out_ch: int, stride: int = 1):
        super().__init__()
        self.conv1 = nn.Conv2d(
            in_ch, out_ch, kernel_size=3, stride=stride, padding=1, bias=False
        )
        self.bn1 = nn.BatchNorm2d(out_ch)
        self.act = nn.ReLU(inplace=True)
        self.conv2 = nn.Conv2d(
            out_ch, out_ch, kernel_size=3, stride=1, padding=1, bias=False
        )
        self.bn2 = nn.BatchNorm2d(out_ch)

        self.shortcut = nn.Identity()
        if stride != 1 or in_ch != out_ch:
            self.shortcut = nn.Sequential(
                nn.Conv2d(in_ch, out_ch, kernel_size=1, stride=stride, bias=False),
                nn.BatchNorm2d(out_ch),
            )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        identity = self.shortcut(x)
        out = self.conv1(x)
        out = self.bn1(out)
        out = self.act(out)
        out = self.conv2(out)
        out = self.bn2(out)
        out = out + identity
        out = self.act(out)
        return out
```

```python
2 usages
class CustomResNet(nn.Module):
    def __init__(self, num_classes: int = 100, blocks_per_stage=(2, 2, 2), base_channels: int = 64):
        super().__init__()
        self.stem = nn.Sequential(
            nn.Conv2d( in_channels: 3, base_channels, kernel_size=3, stride=1, padding=1, bias=False),
            nn.BatchNorm2d(base_channels),
            nn.ReLU(inplace=True),
        )

        self.stage1 = self._make_stage(base_channels, base_channels, blocks_per_stage[0], stride=1)
        self.stage2 = self._make_stage(base_channels, base_channels * 2, blocks_per_stage[1], stride=2)
        self.stage3 = self._make_stage(base_channels * 2, base_channels * 4, blocks_per_stage[2], stride=2)
        self.pool = nn.AdaptiveAvgPool2d((1, 1))
        self.fc = nn.Linear(base_channels * 4, num_classes)

        self._init_weights()

    3 usages
    def _make_stage(self, in_ch: int, out_ch: int, n_blocks: int, stride: int):
        layers = [ResidualBlock(in_ch, out_ch, stride=stride)]
        for _ in range(1, n_blocks):
            layers.append(ResidualBlock(out_ch, out_ch, stride=1))
        return nn.Sequential(*layers)
```

```python
1 usage
def _init_weights(self) -> None:
    for m in self.modules():
        if isinstance(m, nn.Conv2d):
            nn.init.kaiming_normal_(m.weight, mode="fan_out", nonlinearity="relu")
        elif isinstance(m, nn.BatchNorm2d):
            nn.init.constant_(m.weight, val: 1)
            nn.init.constant_(m.bias, val: 0)
        elif isinstance(m, nn.Linear):
            nn.init.normal_(m.weight, mean: 0, std: 0.01)
            nn.init.constant_(m.bias, val: 0)

def forward(self, x: torch.Tensor) -> torch.Tensor:
    x = self.stem(x)
    x = self.stage1(x)
    x = self.stage2(x)
    x = self.stage3(x)
    x = self.pool(x)
    x = torch.flatten(x, 1)
    x = self.fc(x)
    return x
```

Далі створимо файл engine.py, який відповідатиме за навчання та валідацію моделі.

```python
1 usage
def accuracy(logits: torch.Tensor, targets: torch.Tensor) -> float:
    preds = logits.argmax(dim=1)
    return (preds == targets).float().mean().item()
```

```python
def run_epoch(
    model,
    loader,
    criterion,
    optimizer,
    scaler,
    device: torch.device,
    train_mode: bool = True,
) -> Tuple[float, float]:
    if train_mode:
        model.train()
    else:
        model.eval()

    running_loss = 0.0
    running_acc = 0.0
    n_batches = 0

    for x, y in loader:
        x = x.to(device, non_blocking=True)
        y = y.to(device, non_blocking=True)

        if train_mode:
            optimizer.zero_grad(set_to_none=True)

        with torch.set_grad_enabled(train_mode):
            with torch.cuda.amp.autocast(enabled=torch.cuda.is_available()):
                logits = model(x)
                loss = criterion(logits, y)

            if train_mode:
                scaler.scale(loss).backward()
                scaler.step(optimizer)
                scaler.update()

        running_loss += loss.item()
        running_acc += accuracy(logits, y)
        n_batches += 1

    return running_loss / n_batches, running_acc / n_batches
```

І також файл visualize.py для візуалізації графіків.

```python
2 usages
def plot_curves(history):
    plt.figure(figsize=(12, 4))

    plt.subplot(*args: 1, 2, 1)
    plt.plot(*args: history["train_loss"], label="train_loss")
    plt.plot(*args: history["val_loss"], label="val_loss")
    plt.title("Loss")
    plt.xlabel("Epoch")
    plt.legend()

    plt.subplot(*args: 1, 2, 2)
    plt.plot(*args: history["train_acc"], label="train_acc")
    plt.plot(*args: history["val_acc"], label="val_acc")
    plt.title("Accuracy")
    plt.xlabel("Epoch")
    plt.legend()

    plt.tight_layout()
    plt.show()
```

```python
def show_predictions(model, val_loader, classes, device, n_show: int = 8):
    model.eval()

    inv_mean = torch.tensor(MEAN).view(3, 1, 1)
    inv_std = torch.tensor(STD).view(3, 1, 1)

    def denorm(img):
        return torch.clamp(img * inv_std + inv_mean, min: 0, max: 1)

    x_batch, y_batch = next(iter(val_loader))
    x_batch = x_batch.to(device)
    y_batch = y_batch.to(device)

    with torch.no_grad():
        logits = model(x_batch)
        preds = logits.argmax(dim=1)

    plt.figure(figsize=(14, 4))
    for i in range(n_show):
        plt.subplot( *args: 2, 4, i + 1)
        img = denorm(x_batch[i].cpu()).permute(1, 2, 0).numpy()
        plt.imshow(img)
        true_label = classes[y_batch[i].item()]
        pred_label = classes[preds[i].item()]
        color = "green" if true_label == pred_label else "red"
        plt.title( label: f"T: {true_label}\nP: {pred_label}", color=color, fontsize=9)
        plt.axis("off")

    plt.tight_layout()
    plt.show()
```

І основним файлом виступає run_training.py. Тут відбувається побудова, навчання моделі, візуалізація результатів тренування, валідації та роботи.

```python
def main():
    cfg = get_default_cfg()
    set_seed(cfg.seed)

    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    print("PyTorch:", torch.__version__)
    print("CUDA available:", torch.cuda.is_available())
    if torch.cuda.is_available():
        print("GPU:", torch.cuda.get_device_name(0))
    print("Device:", device)

    train_loader, val_loader, classes = build_loaders(cfg)
    num_classes = len(classes)
    print("Train size:", len(train_loader.dataset))
    print("Val size:", len(val_loader.dataset))
    print("Classes:", num_classes)

    model = CustomResNet(
        num_classes=num_classes,
        blocks_per_stage=(2, 2, 2),
        base_channels=64,
    ).to(device)
    print("Trainable params:", sum(p.numel() for p in model.parameters() if p.requires_grad))
```

```python
    history = {
        "train_loss": [],
        "train_acc": [],
        "val_loss": [],
        "val_acc": [],
    }

    best_val_acc = 0.0
    best_path = "best_custom_resnet_cifar100.pt"

    start = time.time()
    for epoch in range(1, cfg.epochs + 1):
        train_loss, train_acc = run_epoch(
            model=model,
            loader=train_loader,
            criterion=criterion,
            optimizer=optimizer,
            scaler=scaler,
            device=device,
            train_mode=True,
        )
        val_loss, val_acc = run_epoch(
            model=model,
            loader=val_loader,
            criterion=criterion,
            optimizer=optimizer,
            scaler=scaler,
            device=device,
            train_mode=False,
        )
        scheduler.step()
```

```python
        history["train_loss"].append(train_loss)
        history["train_acc"].append(train_acc)
        history["val_loss"].append(val_loss)
        history["val_acc"].append(val_acc)

        if val_acc > best_val_acc:
            best_val_acc = val_acc
            torch.save(model.state_dict(), best_path)

        print(
            f"Epoch {epoch:02d}/{cfg.epochs} | "
            f"train_loss={train_loss:.4f} train_acc={train_acc:.4f} | "
            f"val_loss={val_loss:.4f} val_acc={val_acc:.4f}"
        )

    print(f"Finished in {(time.time() - start) / 60:.2f} min")
    print(f"Best val acc: {best_val_acc:.4f} | saved: {best_path}")

    model.load_state_dict(torch.load(best_path, map_location=device))
    plot_curves(history)
    show_predictions(model, val_loader, classes, device=device, n_show=8)


if __name__ == "__main__":
    main()
```

Так як є наявність CUDA локально, то краще її використовувати задля кращої роботи під час навчання.

```
(.venv) PS D:\KPI\5course2semestr\TMN\Labs\L1\code> python -c "import torch; print(torch.__version__); print(torch.cuda.is_available()); print(torch.version.cuda); print(torch.cuda.get_device_name(0) if torch.cuda.is_available
 else 'NO GPU')"
2.5.1+cu121
True
12.1
NVIDIA GeForce GTX 1650
```

Запустимо файл run_training.py.

Спочатку йде повторна перевірка на наявність CUDA:

```
PyTorch: 2.5.1+cu121
CUDA available: True
GPU: NVIDIA GeForce GTX 1650
Device: cuda
```

Далі завантажується датасет CIFAR 100:

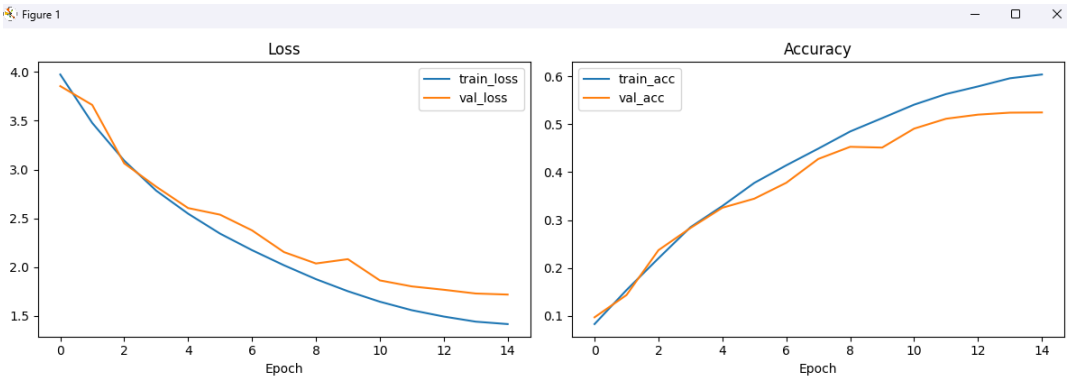
```
Device: cuda
Downloading https://www.cs.toronto.edu/~kriz/cifar-100-python.tar.gz to ./data\cifar-100-python.tar.gz
100.0%
Extracting ./data\cifar-100-python.tar.gz to ./data
```

```
Files already downloaded and verified
Train size: 30000
Val size: 6000
Classes: 100
D:\KPI\5course2semestr\TMN\Labs\L1\code\run_training.py:41: FutureWarning: `torch.cuda.amp.GradScaler(args...)` is deprecated. Please use `torch.amp.GradScaler('cuda', args...)` instead.
  scaler = torch.cuda.amp.GradScaler(enabled=torch.cuda.is_available())
Trainable params: 2800804
D:\KPI\5course2semestr\TMN\Labs\L1\code\src\engine.py:36: FutureWarning: `torch.cuda.amp.autocast(args...)` is deprecated. Please use `torch.amp.autocast('cuda', args...)` instead.
  with torch.cuda.amp.autocast(enabled=torch.cuda.is_available()):
```

Відбувається тренуванні моделі:

```
Epoch 01/15 | train_loss=3.9744 train_acc=0.0828 | val_loss=3.8552 val_acc=0.0969
Epoch 02/15 | train_loss=3.4778 train_acc=0.1538 | val_loss=3.6622 val_acc=0.1434
Epoch 03/15 | train_loss=3.0917 train_acc=0.2201 | val_loss=3.0635 val_acc=0.2370
Epoch 04/15 | train_loss=2.7838 train_acc=0.2850 | val_loss=2.8242 val_acc=0.2833
Epoch 05/15 | train_loss=2.5473 train_acc=0.3293 | val_loss=2.6049 val_acc=0.3257
Epoch 06/15 | train_loss=2.3427 train_acc=0.3775 | val_loss=2.5371 val_acc=0.3446
Epoch 07/15 | train_loss=2.1732 train_acc=0.4142 | val_loss=2.3761 val_acc=0.3778
Epoch 08/15 | train_loss=2.0181 train_acc=0.4491 | val_loss=2.1533 val_acc=0.4275
Epoch 09/15 | train_loss=1.8759 train_acc=0.4849 | val_loss=2.0366 val_acc=0.4529
Epoch 10/15 | train_loss=1.7521 train_acc=0.5128 | val_loss=2.0809 val_acc=0.4513
Epoch 11/15 | train_loss=1.6447 train_acc=0.5409 | val_loss=1.8630 val_acc=0.4908
Epoch 12/15 | train_loss=1.5574 train_acc=0.5631 | val_loss=1.8015 val_acc=0.5115
Epoch 13/15 | train_loss=1.4924 train_acc=0.5789 | val_loss=1.7672 val_acc=0.5201
Epoch 14/15 | train_loss=1.4400 train_acc=0.5960 | val_loss=1.7287 val_acc=0.5242
D:\KPI\5course2semestr\TMN\Labs\L1\code\run_training.py:93: FutureWarning: You are using `to
  model.load_state_dict(torch.load(best_path, map_location=device))
Epoch 15/15 | train_loss=1.4152 train_acc=0.6039 | val_loss=1.7179 val_acc=0.5246
Finished in 30.43 min
Best val acc: 0.5246 | saved: best_custom_resnet_cifar100.pt
```

Графіки:

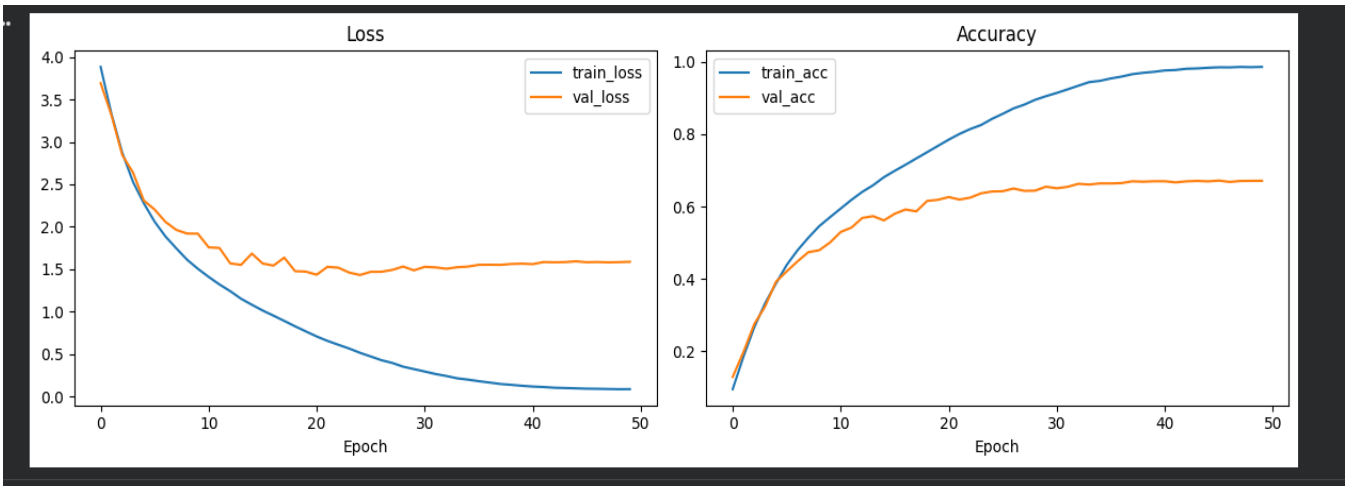


Результати розпізнавання:

Незважаючи на те, що є помилки в розпізнаванні, все ж результат навчання, валідації моделі є досить непоганим, що можна підтвердити графіками втрат та точності. Втрати знижаються з кожною новою епохою, а точність навпаки росте. Так як використовували 0.6 датасету на 15 епохах, кінцевий результат val_acc = 0.5246 є досить непоганим. Якщо збільшити кількість епох, результат може значно покращитись.

Спробуємо ще один тест з 50 епохами та 0.8 датасету. Для швидшого виконання код був сконвертований в Ipynb задля виконання на потужних ГП в Google Colab. Отримані результати:

```
Epoch 25/50 | train_loss=0.5153 train_acc=0.8421 | val_loss=1.4313 val_acc=0.6416
Epoch 26/50 | train_loss=0.4715 train_acc=0.8561 | val_loss=1.4685 val_acc=0.6424
Epoch 27/50 | train_loss=0.4276 train_acc=0.8711 | val_loss=1.4693 val_acc=0.6498
Epoch 28/50 | train_loss=0.3953 train_acc=0.8816 | val_loss=1.4916 val_acc=0.6434
Epoch 29/50 | train_loss=0.3516 train_acc=0.8949 | val_loss=1.5309 val_acc=0.6440
Epoch 30/50 | train_loss=0.3230 train_acc=0.9049 | val_loss=1.4865 val_acc=0.6549
Epoch 31/50 | train_loss=0.2943 train_acc=0.9138 | val_loss=1.5274 val_acc=0.6505
Epoch 32/50 | train_loss=0.2651 train_acc=0.9236 | val_loss=1.5210 val_acc=0.6544
Epoch 33/50 | train_loss=0.2416 train_acc=0.9338 | val_loss=1.5052 val_acc=0.6628
Epoch 34/50 | train_loss=0.2143 train_acc=0.9437 | val_loss=1.5227 val_acc=0.6608
Epoch 35/50 | train_loss=0.1989 train_acc=0.9472 | val_loss=1.5302 val_acc=0.6639
Epoch 36/50 | train_loss=0.1805 train_acc=0.9538 | val_loss=1.5512 val_acc=0.6638
Epoch 37/50 | train_loss=0.1644 train_acc=0.9590 | val_loss=1.5523 val_acc=0.6648
Epoch 38/50 | train_loss=0.1475 train_acc=0.9656 | val_loss=1.5506 val_acc=0.6698
Epoch 39/50 | train_loss=0.1374 train_acc=0.9694 | val_loss=1.5611 val_acc=0.6687
Epoch 40/50 | train_loss=0.1264 train_acc=0.9721 | val_loss=1.5649 val_acc=0.6699
Epoch 41/50 | train_loss=0.1172 train_acc=0.9760 | val_loss=1.5593 val_acc=0.6699
Epoch 42/50 | train_loss=0.1116 train_acc=0.9772 | val_loss=1.5839 val_acc=0.6668
Epoch 43/50 | train_loss=0.1032 train_acc=0.9806 | val_loss=1.5812 val_acc=0.6696
Epoch 44/50 | train_loss=0.0997 train_acc=0.9815 | val_loss=1.5836 val_acc=0.6709
Epoch 45/50 | train_loss=0.0961 train_acc=0.9833 | val_loss=1.5927 val_acc=0.6695
Epoch 46/50 | train_loss=0.0920 train_acc=0.9846 | val_loss=1.5818 val_acc=0.6718
Epoch 47/50 | train_loss=0.0907 train_acc=0.9844 | val_loss=1.5842 val_acc=0.6680
Epoch 48/50 | train_loss=0.0883 train_acc=0.9856 | val_loss=1.5804 val_acc=0.6706
Epoch 49/50 | train_loss=0.0860 train_acc=0.9850 | val_loss=1.5829 val_acc=0.6710
Epoch 50/50 | train_loss=0.0869 train_acc=0.9857 | val_loss=1.5871 val_acc=0.6711
Finished in 16.52 min
Best val acc: 0.6718 | saved: best_custom_resnet_cifar100.pt
```



Як видно, результат став трішки краще за рахунок збільшення кількості тренувальних та валідаційних даних та кількості епох. Помилок розпізнавання зображень стало трішки менше.

В рамках завдання 2 треба експортувати модель в ONNX, та створити API з ендпоінтом /predict який прийматиме на вхід зображення та повертатиме результати розпізнавання.

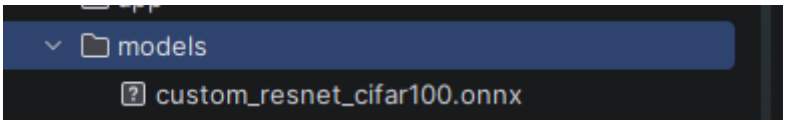
Для цього створимо нову директорію “api” на рівні з минулою “code”.

Далі створимо файл export_to_onnx:

```python
def export_onnx(
    checkpoint_path: str = "../code/best_custom_resnet_cifar100.pt",
    onnx_path: str = "models/custom_resnet_cifar100.onnx",
):
    ckpt = Path(checkpoint_path)
    if not ckpt.exists():
        raise FileNotFoundError(
            f"Checkpoint not found at '{ckpt}'. "
            "Train the model first or provide a correct path."
        )

    onnx_file = Path(onnx_path)
    onnx_file.parent.mkdir(parents=True, exist_ok=True)

    model = CustomResNet(num_classes=100, blocks_per_stage=(2, 2, 2), base_channels=64)
    state = torch.load(ckpt, map_location="cpu", weights_only=True)
    model.load_state_dict(state)
    model.eval()

    dummy_input = torch.randn(1, 3, 32, 32, dtype=torch.float32)
    torch.onnx.export(
        model,
        dummy_input,
        str(onnx_file),
        export_params=True,
        opset_version=13,
        do_constant_folding=True,
        input_names=["input"],
        output_names=["logits"],
        dynamic_axes={"input": {0: "batch_size"}, "logits": {0: "batch_size"}},
    )
    print(f"ONNX model exported to: {onnx_file}")


if __name__ == "__main__":
    export_onnx()
```

Тут на вхід приймається шлях до тренованої моделі (файл .pt) та шлях виводу моделі в форматі .onnx.

Запустивши даний файл отримаємо в піддиректорії models готову модель:



Далі створимо саму API яка буде працювати з моделлю. Для цього в піддиректорії app створимо такі файли.

Preprocessing:

```python
CIFAR100_MEAN = np.array( object: [0.5071, 0.4867, 0.4408], dtype=np.float32)
CIFAR100_STD = np.array( object: [0.2675, 0.2565, 0.2761], dtype=np.float32)


2 usages
def preprocess_image(image_bytes: bytes) -> np.ndarray:
    image = Image.open(BytesIO(image_bytes)).convert("RGB")
    image = image.resize( size: (32, 32), Image.BILINEAR)
    arr = np.asarray(image, dtype=np.float32) / 255.0
    arr = (arr - CIFAR100_MEAN) / CIFAR100_STD
    arr = np.transpose(arr,  axes: (2, 0, 1))
    arr = np.expand_dims(arr, axis=0)
    return arr.astype(np.float32)
```

ONNXPredictor:

```python
class ONNXPredictor:
    def __init__(self, model_path: str):
        model_file = Path(model_path)
        if not model_file.exists():
            raise FileNotFoundError(
                f"ONNX model not found at '{model_file}'. "
                "Run export_to_onnx.py first."
            )

        providers = ["CPUExecutionProvider"]
        self.session = ort.InferenceSession(str(model_file), providers=providers)
        self.input_name = self.session.get_inputs()[0].name
        self.output_name = self.session.get_outputs()[0].name

    def predict(self, input_tensor: np.ndarray, top_k: int = 5):
        logits = self.session.run( output_names: [self.output_name],  input_feed: {self.input_name: input_tensor})[0]
        logits = logits[0]

        exp = np.exp(logits - np.max(logits))
        probs = exp / np.sum(exp)

        top_idx = np.argsort(-probs)[:top_k]
        top = [
            {
                "class_id": int(i),
                "label": CIFAR100_CLASSES[int(i)],
                "probability": float(probs[int(i)]),
            }
            for i in top_idx
        ]

        return {
            "top1": top[0],
            "top_k": top,
        }
```

І також основний файл з ендпоінтами і логікою:

```python
import os

from fastapi import FastAPI, File, HTTPException, UploadFile

from .onnx_predictor import ONNXPredictor
from .preprocessing import preprocess_image


MODEL_PATH = os.getenv("MODEL_PATH", "models/custom_resnet_cifar100.onnx")

app = FastAPI(title="CIFAR-100 Inference API", version="1.0.0")
predictor = ONNXPredictor(model_path=MODEL_PATH)

@app.post("/predict")
async def predict(file: UploadFile = File(...)):
    if not file.content_type or not file.content_type.startswith("image/"):
        raise HTTPException(status_code=400, detail="Uploaded file must be an image.")

    image_bytes = await file.read()
    if not image_bytes:
        raise HTTPException(status_code=400, detail="Empty file.")

    input_tensor = preprocess_image(image_bytes)
    pred = predictor.predict(input_tensor=input_tensor, top_k=5)
    return pred
```

І також Dockerfile для збирання моделі, апі в єдиний імедж:

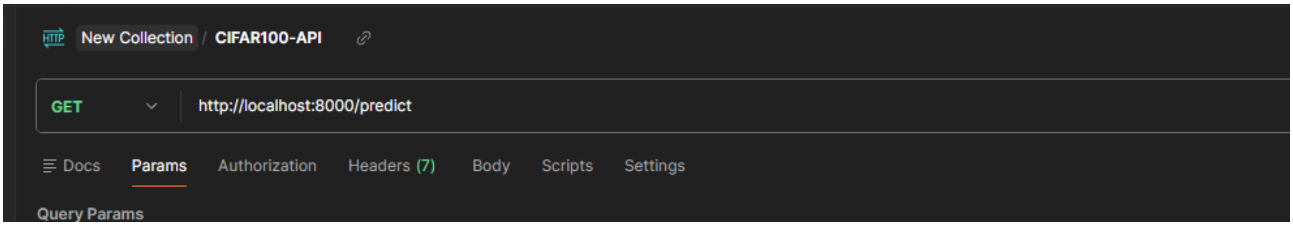
```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY app ./app
COPY models ./models

ENV MODEL_PATH=models/custom_resnet_cifar100.onnx
EXPOSE 8000

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

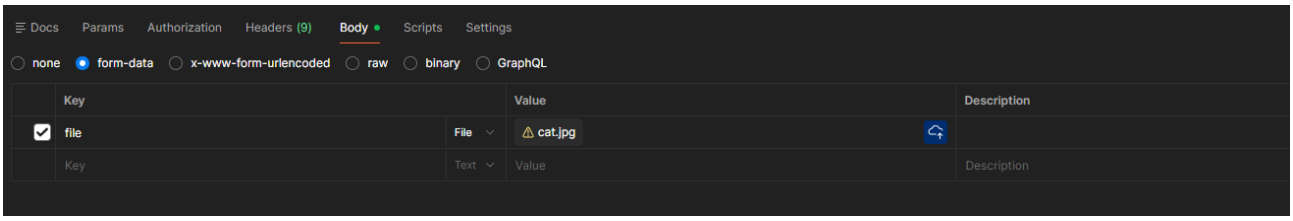
Спробуємо підняти контейнер:

```
PS D:\KPI\5course2semestr\TMN\Labs\L1> docker build -t cifar100-api ./api
[+] Building 51.1s (11/11) FINISHED                                                                                    docker:desktop-linu
 => [internal] load build definition from Dockerfile                                                                                   0.0
 => => transferring dockerfile: 323B                                                                                                   0.0
 => [internal] load metadata for docker.io/library/python:3.11-slim                                                                    1.5
 => [internal] load .dockerignore                                                                                                      0.1
 => => transferring context: 89B                                                                                                       0.0
 => [1/6] FROM docker.io/library/python:3.11-slim@sha256:d6e4d224f70f9e0172a06a3a2eba2f768eb146811a349278b38fff3a36463b47              6.5
 => => resolve docker.io/library/python:3.11-slim@sha256:d6e4d224f70f9e0172a06a3a2eba2f768eb146811a349278b38fff3a36463b47              0.0
 => => sha256:206356c42440674ecbdf1070cf70ce8ef7885ac2e5c56f1ecf800b758f6b0419 29.78MB / 29.78MB                                       2.0
 => => sha256:20670944442553bb5bf0f8b525f10409691c57f48c48e6bebac8c3a85df4f9dd2 1.29MB / 1.29MB                                        0.6
 => => sha256:bbf418262cbe98089272fafddcc570e7fa85e6fa28fe7163dc445136042bd183 14.37MB / 14.37MB                                       1.5
 => => sha256:43272e9c9d4bd6009934139dd3c958777b55bdbe0ccadb5e7a13f847436fabd2 250B / 250B                                            0.4
 => => extracting sha256:206356c42440674ecbdf1070cf70ce8ef7885ac2e5c56f1ecf800b758f6b0419                                              2.4
 => => extracting sha256:20670944442553bb5bf0f8b525f10409691c57f48c48e6bebac8c3a85df4f9dd2                                              0.3
 => => extracting sha256:bbf418262cbe98089272fafddcc570e7fa85e6fa28fe7163dc445136042bd183                                              1.4
 => => extracting sha256:43272e9c9d4bd6009934139dd3c958777b55bdbe0ccadb5e7a13f847436fabd2                                              0.0
 => [internal] load build context                                                                                                      0.7
 => => transferring context: 11.23MB                                                                                                   0.5
 => [2/6] WORKDIR /app                                                                                                                 0.3
 => [3/6] COPY requirements.txt .                                                                                                      0.0
 => [4/6] RUN pip install --no-cache-dir -r requirements.txt                                                                          25.0
 => [5/6] COPY app ./app                                                                                                               0.1
 => [6/6] COPY models ./models                                                                                                         0.1
 => exporting to image                                                                                                                17.0
 => => exporting layers                                                                                                                9.8
 => => exporting manifest sha256:80edb5c2f515eda2502af1f0ad29e2134fc472415f87dfa0cb9d8033ee25d7fe                                      0.0
 => => exporting config sha256:99111bdf1efb70ad3c75dd5c8786aa0595481776 6ac2f51791d152ba73d096c6                                      0.0
 => => exporting attestation manifest sha256:f38b4971b9aafd10aeb07342fa4c1b668b383898a10f5c805049957ca266f32b                          0.0
 => => exporting manifest list sha256:af1f142d8aae90ebb53122f4750964619dcb4b258e375f194bbaf70c4cf30c24                                 0.0
 => => naming to docker.io/library/cifar100-api:latest                                                                                 0.0
 => => unpacking to docker.io/library/cifar100-api:latest                                                                              7.1
```

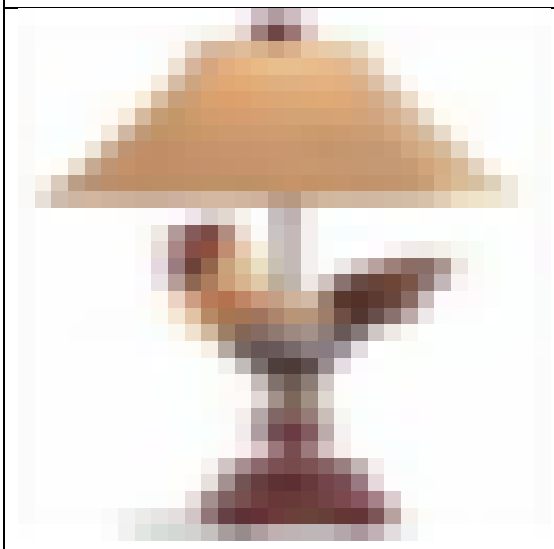
```
PS D:\KPI\5course2semestr\TMN\Labs\L1> docker run --rm -p 8000:8000 cifar100-api
INFO:     Started server process [1]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
```

Тепер спробуємо затестувати ендпоінт:

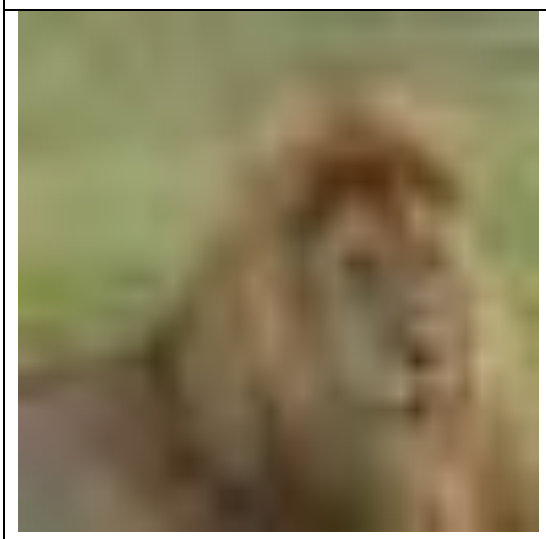




Картинка	Відповідь АПІ

	{
	"top1": {
	"class_id": 40,
	"label": "lamp",
	"probability": 0.8734777569770813
	},
	"top_k": [
	{
	"class_id": 40,
	"label": "lamp",
	"probability": 0.8734777569770813
	},
	{
	"class_id": 28,
	"label": "cup",
	"probability": 0.03396232798695564
	},
	{
	"class_id": 51,
	"label": "mushroom",
	"probability": 0.030433084815740585
	},
	{
	"class_id": 20,
	"label": "chair",
	"probability": 0.01961992122232914
	},
	{
	"class_id": 46,
	"label": "man",
	"probability": 0.005822116043418646
	}
	]
	}

	{
	"top1": {
	"class_id": 0,
	"label": "apple",
	"probability": 0.49889349937438965
	},
	"top_k": [
	{
	"class_id": 0,
	"label": "apple",
	"probability": 0.49889349937438965
	},
	{
	"class_id": 57,
	"label": "pear",
	"probability": 0.3457542657852173
	},
	{
	"class_id": 53,
	"label": "orange",
	"probability": 0.09217512607574463
	},
	{
	"class_id": 83,
	"label": "sweet_pepper",
	"probability": 0.04557335749268532
	},
	{
	"class_id": 92,
	"label": "tulip",
	"probability": 0.007781886029988527
	}
	]
	}




```
{
    "top1": {
        "class_id": 43,
        "label": "lion",
        "probability": 0.879926323890686
    },
    "top_k": [
        {
            "class_id": 43,
            "label": "lion",
            "probability": 0.879926323890686
        },
        {
            "class_id": 38,
            "label": "kangaroo",
            "probability": 0.051385410130023956
        },
        {
            "class_id": 80,
            "label": "squirrel",
            "probability": 0.011842350475490093
        },
        {
            "class_id": 42,
            "label": "leopard",
            "probability": 0.011488175019621849
        },
        {
            "class_id": 15,
            "label": "camel",
            "probability": 0.011036938056349754
        }
    ]
}
```

```
{
    "top1": {
        "class_id": 81,
        "label": "streetcar",
        "probability": 0.8702906370162964
    },
    "top_k": [
        {
            "class_id": 81,
            "label": "streetcar",
            "probability": 0.8702906370162964
        },
        {
            "class_id": 90,
            "label": "train",
            "probability": 0.08012933284044266
        },
        {
            "class_id": 13,
            "label": "bus",
            "probability": 0.04300154000520706
        },
        {
            "class_id": 58,
            "label": "pickup_truck",
            "probability": 0.0028533916920423508
        },
        {
            "class_id": 37,
            "label": "house",
            "probability": 0.0026735463179647923
        }
    ]
}
```

Як бачимо, якщо використовувати примітивні зображення (схожі на ті, на яких модель тренувалась та валідувалась) то модель коректно розпізнає клас з досить високою впевненістю. Якщо використовувати складніші зображення (напр. більший розмір, або декілька об’єктів на фото, і тп.) то модель може неправильно розпізнавати, або розпізнавати з меншою впевненістю.

Висновки: було реалізовано цикл створення та підготовки нейромережевої моделі. Для задачі класифікації обрано набір даних CIFAR-100 та архітектуру ResNet, яку реалізовано з нуля без використання готових pretrained-моделей. Було підготовлено пайплайн завантаження і попередньої обробки даних, налаштовано навчання моделі з урахуванням обмежень обчислювальних ресурсів, проведено тренування та оцінювання якості. За результатами експериментів модель продемонструвала стабільне навчання та досягла адекватної точності для обраної конфігурації, а також були отримані коректні приклади передбачень на тестових зображеннях. На другому етапі модель була підготовлена до розгортання: виконано конвертацію у формат ONNX, реалізовано inference API на FastAPI з endpoint /predict, що приймає зображення та повертає прогноз класу з імовірностями, а також створено Docker-конфігурацію для контейнеризованого запуску сервісу. Додатково перевірено працездатність API на тестових зображеннях і сформовано приклади успішних передбачень для демонстрації результату. Також було отримано розуміння впливу обсягу даних, кількості епох і обчислювальних ресурсів на якість моделі та швидкість експериментів, а також важливості перевірки моделі на даних, близьких до тренувального розподілу.